INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 5: Manejo de ficheros en Java (creación, escritura,consulta de metadatos y eliminación de archivos)
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Vayas Andres
Asignatura:	Lógica y programación
Docente:	Ing. José Rubén Caiza Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar programas en Java que permitan crear, escribir, consultar y eliminar ficheros de texto, aplicando las clases File, FileWriter y BufferedWriter y el manejo de excepciones de entrada y salida.

Específicos:

- Crear, comprobar y eliminar ficheros con la clase File (exists, createNewFile, delete,getName y getPath).
- Almacenar información por teclado en archivos de texto con FileWriter y BufferedWriter, tanto en modo sobrescritura como en modo append.
- Controlar errores con try-catch (IOException y NumberFormatException) y liberar los recursos con close() o try-with-resources.

2.2 Modalidad

Detallada en la guía.

2.3 Tiempo de duración

Presenciales: Detallado en la guía
No presenciales: Detallado en la guía

2.4 Instrucciones

Detalladas en la guía proporcionada por el docente.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computador personal con acceso a internet.
- Java Development Kit (JDK) y un entorno de desarrollo (IDE) o editor para Java.
- Cuenta de GitHub para el repositorio de los ejercicios.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☐Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): GitHub (repositorio de código) y entorno de desarrollo Java

2.6 Actividades por desarrollar

Detalladas en la guía proporcionada por el docente.

2.7 Resultados obtenidos

Parte 1: APE 5 primera parte

Ejercicio 1. Crear un fichero en Java

Enunciado: Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Código:

```java
import java.io.File;
import java.io.IOException;

public class Ejercicio1 {
    public static void main(String[] args) {
        try {
            File archivo = new File("curso.txt");

            if (archivo.createNewFile()) {
                System.out.println("Archivo creado correctamente: "
                        + archivo.getName());
            } else {
                System.out.println("El archivo ya existe: "
                        + archivo.getName());
            }
        } catch (IOException e) {
            System.out.println("Ocurrió un error al crear el archivo: "
                    + e.getMessage());
        }
    }
}
```

Figura 1. Código fuente del Ejercicio 1 (Ejerciciol.java).

Resultado esperado:

Primera ejecución: Archivo creado correctamente: curso.txt

Siguientes ejecuciones: El archivo ya existe: curso.txt

Ejercicio 2. Escribir información en un fichero

Enunciado: Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Nota: el archivo estudiantes.txt debe existir previamente.

Código:

```java
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio2 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Escribe un mensaje: ");
        String mensaje = teclado.nextLine();

        try {
            // El parámetro true conserva la información anterior
            FileWriter escritor = new FileWriter("estudiantes.txt", true);
            escritor.write(mensaje + System.lineSeparator());
            escritor.close();

            System.out.println("La información se guardó correctamente.");
        } catch (IOException e) {
```

```
            System.out.println("Ocurrió un error al escribir en el archivo: "
                    + e.getMessage());
        }

        teclado.close();
    }
}
```

Figura 2. Código fuente del Ejercicio 2 (Ejercicio2.java).

Resultado esperado:

· Se solicita el mensaje por teclado, se agrega al final de estudiantes.txt y se muestra:La información se guardó correctamente.

Ejercicio 3. Eliminar un fichero en Java

Enunciado: Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Código:

```
import java.io.File;

public class Ejercicio3 {
    public static void main(String[] args) {
        File archivo = new File("estudiantes.txt");

        if (archivo.delete()) {
            System.out.println("El archivo " + archivo.getName()
                    + " se eliminó correctamente.");
        } else {
            System.out.println("No se pudo eliminar el archivo. "
                    + "Verifica que exista y que no esté en uso.");
        }
    }
}
```

Figura 3. Código fuente del Ejercicio 3 (Ejercicio3.java).

Resultado esperado:

· Si el archivo existe: El archivo estudiantes.txt se eliminó correctamente.

· Si no existe o no se puede borrar: se muestra el mensaje de error.

Parte 2: Ejercicios de APE clase 2 (05/11/2026)

Ejercicio **4. Registro de estudiantes y calificaciones**

Enunciado: Desarrollar un programa en Java que permita registrar los datos de 3 estudiantes y guardar la información en un archivo llamado estudiantes.txt.

Requisitos: solicitar nombre y calificación, guardar ambos datos con cada estudiante en dos lineas, usar Scanner, BufferedWriter y FileWriter, y cerrar correctamente el archivo.

Código:

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio4 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // try-with-resources: cierra el archivo automáticamente al terminar
        try (BufferedWriter escritor =
                new BufferedWriter(new FileWriter("estudiantes.txt"))) {

            for (int i = 1; i <= 3; i++) {
                System.out.print("Nombre del estudiante " + i + ": ");
                String nombre = teclado.nextLine();

```

```
                System.out.print("Calificación de " + nombre + ": ");
                double calificacion = Double.parseDouble(teclado.nextLine());

                // Cada estudiante ocupa dos líneas
                escritor.write(nombre);
                escritor.newLine();
                escritor.write(String.valueOf(calificacion));
                escritor.newLine();
            }

            System.out.println("Datos guardados en estudiantes.txt");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("La calificación debe ser un número válido.");
        }

        teclado.close();
    }
}
```

Figura 4. Código fuente del Ejercicio 4 (Ejercicio4.java).

Resultado esperado:

- El archivo estudiantes.txt queda con 6 líneas: nombre y calificación de cada uno de los 3 estudiantes.

Ejercicio 5. Registro de productos con modo Append

Enunciado: Desarrollar un programa que permita registrar productos en un archivo llamado productos.txt.

Requisitos: solicitar nombre, precio y cantidad; calcular el valor total(precio x cantidad);guardar los resultados sin borrar los registros anteriores; usar BufferedWriter y modo append.

Código:

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio5 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String continuar;

        // El segundo parámetro true activa el modo append
        try (BufferedWriter escritor =
                new BufferedWriter(new FileWriter("productos.txt", true))) {

            do {
                System.out.print("Nombre del producto: ");
                String nombre = teclado.nextLine();

                System.out.print("Precio: ");
                double precio = Double.parseDouble(teclado.nextLine());

                System.out.print("Cantidad disponible: ");
                int cantidad = Integer.parseInt(teclado.nextLine());

                double total = precio * cantidad;

                escritor.write(String.format(Locale.US,
                    "Producto: %s | Precio: $%.2f | Cantidad: %d | Total: $%.2f",
                    nombre, precio, cantidad, total));
                escritor.newLine();

                System.out.print("¿Registrar otro producto? (si/no): ");
                continuar = teclado.nextLine();
            } while (continuar.equalsIgnoreCase("si"));
```

```
36
37              System.out.println("Productos guardados en productos.txt");
38          } catch (IOException e) {
39              System.out.println("Error al escribir el archivo: " + e.getMessage());
40          } catch (NumberFormatException e) {
41              System.out.println("Precio o cantidad no válidos.");
42          }
43
44          teclado.close();
45      }
46  }
```

Figura 5. Código fuente del Ejercicio 5 (Ejercicio5.java).

Resultado esperado:

- Cada producto se agrega como una línea nueva en productos.txt, con su valor total calculado.
- Al ejecutarde nuevo el programa, los registros anteriores se conservan.

Ejercicio 6. Sistema de registro de ventas

Enunciado: Desarrollar un programa en Java que permita registrar las ventas de una tienda y guardar la información en un archivo llamado ventas.txt.

Requisitos: solicitar producto, precio y cantidad; calcular el subtotal; aplicar 10% de descuento si el subtotal es mayor o igual a $100; mostrar el total a pagar; repetir hasta que se escriba fin; guardar cada venta sin eliminar las anteriores y, al final, guardar el total acumulado.

Código:

```
1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4  import java.util.Locale;
5  import java.util.Scanner;
6
7  public class Ejercicio6 {
8      public static void main(String[] args) {
9          Scanner teclado = new Scanner(System.in);
10         double totalAcumulado = 0;
11
12         try (BufferedWriter escritor =
13                 new BufferedWriter(new FileWriter("ventas.txt", true))) {
14
15             while (true) {
16                 System.out.print("Producto (escribe fin para terminar): ");
17                 String nombre = teclado.nextLine();
18                 if (nombre.equalsIgnoreCase("fin")) {
19                     break;
20                 }
21
22                 System.out.print("Precio: ");
23                 double precio = Double.parseDouble(teclado.nextLine());
24
25                 System.out.print("Cantidad vendida: ");
26                 int cantidad = Integer.parseInt(teclado.nextLine());
27
28                 double subtotal = precio * cantidad;
29                 double descuento = 0;
30
31                 if (subtotal >= 100) {
32                     descuento = subtotal * 0.10;
33                 }
34
35                 double totalPagar = subtotal - descuento;
36                 totalAcumulado += totalPagar;
37
38                 System.out.printf(Locale.US, "Total a pagar: $%.2f%n", totalPagar);
39
40                 escritor.write(String.format(Locale.US,
41                     "Producto: %s | Subtotal: $%.2f | Descuento: $%.2f"
```

```
                    + " | Total: $%.2f",
                    nombre, subtotal, descuento, totalPagar));
                escritor.newLine();
            }

            escritor.write(String.format(Locale.US,
                "TOTAL ACUMULADO DE VENTAS: $%.2f", totalAcumulado));
            escritor.newLine();

            System.out.printf(Locale.US,
                "Total acumulado: $%.2f%n", totalAcumulado);
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Precio o cantidad no válidos.");
        }

        teclado.close();
    }
}
```

Figura 6. Código fuente del Ejercicio 6 (Ejercicio6.java)

Resultado esperado:

- Ejemplo: 3 unidades a $40 dan subtotal $120.00, descuento $12.00 y total a pagar $108.00.
- Cada venta se guarda en ventas.txt y al final se agrega la línea con el total acumulado.

Parte 3: Ejercicios APE 5 (segunda parte)

Ejercicio 7. Exploración y gestión de metadatos con File

Enunciado: Desarrollar un programa en Java que use la clase File para interactuar con la ruta "datos/prueba.txt".

Requisitos: comprobar con exists(si el recurso existe; si no existe, crearlo con createNewFile() manejando IOException con try-catch; y mostrar por consola getName() y getPath(), tanto si ya existía como si se acaba de crear.

Código:

```
import java.io.File;
import java.io.IOException;

public class Ejercicio7 {
    public static void main(String[] args) {
        File archivo = new File("datos/prueba.txt");

        // 1. Comprobación previa
        if (archivo.exists()) {
            System.out.println("El archivo ya existe.");
        } else {
            try {
                // La carpeta "datos" debe existir; si no, se crea
                archivo.getParentFile().mkdirs();

                // 2. Creación controlada
                if (archivo.createNewFile()) {
                    System.out.println("Archivo creado correctamente.");
                }
            } catch (IOException e) {
                System.out.println("Error de E/S al crear el archivo: "
                        + e.getMessage());
            }
        }

        // 3. Extracción de información
        System.out.println("Nombre: " + archivo.getName());
        System.out.println("Ruta: " + archivo.getPath());
    }
}
```

Figura 7. Código fuente del Ejercicio 7 (Ejercicio7.java).

Resultado esperado:

Primera ejecución: Archivo creado correctamente. / Nombre: prueba.txt / Ruta:datos/prueba.txt (en Windows: datos\prueba.txt).

· Siguientes ejecuciones: El archivo ya existe. y se muestran el mismo nombre y ruta.

Ejercicio 8. Escritura estructurada por líneas con BufferedWriter

Enunciado: Desarrollar un programa en Java que cree un fichero de texto llamado materias.txt y escriba en él cinco asignaturas.

Requisitos: usar BufferedWriter enlazado con FileWriter, newLine() para una asignatura por línea, cerrar el flujo con close() y controlar IOException.

Código:

```java
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio8 {
    public static void main(String[] args) {
        String[] materias = {
            "Programación Orientada a Objetos",
            "Cálculo Diferencial",
            "Estructuras de Datos",
            "Bases de Datos",
            "Fundamentos de Redes"
        };

        BufferedWriter escritor = null;
        try {
            escritor = new BufferedWriter(new FileWriter("materias.txt"));

            for (String materia : materias) {
                escritor.write(materia);
                escritor.newLine(); // una asignatura por línea
            }
            System.out.println("materias.txt creado correctamente.");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } finally {
            // Liberar recursos con close()
            try {
                if (escritor != null) {
                    escritor.close();
                }
            } catch (IOException e) {
                System.out.println("Error al cerrar el archivo: " + e.getMessage());
            }
        }
    }
}
```

Figura 8. Código fuente del Ejercicio 8 (Ejercicio8,java).

Resultado esperado:

· Se genera materias.txt en la carpeta del proyecto con 5 líneas, una asignatura por línea.

Verificación: abrir el archivo y comprobar que cada asignatura está en una línea independiente.

Ejercicio 9. Control del modo de apertura (append vs sobrescribir)

Enunciado: A partir del archivo materias.txt del ejercicio anterior, desarrollar un programa que añada dos asignaturas nuevas al final sin borrar el contenido previo.

Requisitos: abrir FileWriter con append = true, usar newLine() para mantener el formato de líneas y cerrar el flujo con close(.

Código:

```
1 import java.io.BufferedWriter;
2 import java.io.FileWriter;
3 import java.io.IOException;
4
5 public class Ejercicio9 {
6       public static void main(String[] args) {
7             BufferedWriter escritor = nu11;
8             try {
9                   // EL segundo parámetro true activa el modo append
10                  escritor = new BufferedWriter(new FileWriter("materias.txt", true));
11
12                  escritor.write("Ingeniería de Software");
13                  escritor.newLine();
14                  escritor.write("Sistemas Operativos");
15                  escritor.newLine();
16
17                  System.out.println("Asignaturas añadidas al final de materias.txt");
18            } catch (IOException e) {
19                  System.out.println("Error al escribir el archivo: " + e.getMessage());
20            } finally {
21                  try {
22                        if (escritor != null) {
23                              escritor.close();
24                        }
25                  } catch (IOException e) {
26                        System.out.println("Error al cerrar el archivo: " + e.getMessage());
27                  }
28            }
29      }
30 }
```

Figura 9. Código fuente del Ejercicio 9 (Ejercicio9.java).

Resultado esperado:

- materias.txt conserva las 5 asignaturas originales y ahora tiene 2 nuevas al final (7líneas en total).
- Si se usara FileWriter sin el parámetro true, el contenido anterior se sobrescribiría.

Ejercicio 10. Ciclo de vida completo y borrado seguro

Enunciado: Desarrollar un programa en Java que gestione el ciclo de vida del archivo "datos/temporal.txt": comprobar si existe, crearlo si hace falta y eliminarlo solo si el usuario confirma.

*Requisitos: crear un objeto File con la ruta, usar exists(y createNewFile(controlando IOException, declarar boolean confirmarBorrado*ρ*=tu para simular la confirmación, y usar delete() solo si es verdadera mostrando su resultado.*

Código:

```
1 import java.io.File;
2 import java.io.IOException;
3
  public class Ejercicio10 {
4       public static void main(String[] args) {
5             File archivo = new File("datos/temporal.txt");
6
7             // Consulta del estado y creación segura
8             if (!archivo.exists()) {
9                   try {
10                        archivo.getParentFile().mkdirs(); // asegura La carpeta "datos"
11                        archivo.createNewFile();
12                        System.out.println("Archivo creado: " + archivo.getPath());
13                  } catch (IOException e) {
14                        System.out.println("Error de E/S al crear el archivo: "
15                                    + e.getMessage());
16                  }
17            } else {
18                  System.out.println("E1 archivo ya existe: " + archivo.getPath());
19            }
20
```

```
21
22          // Validación: simula la confirmación del usuario
23          boolean confirmarBorrado = true;
24
25          if (confirmarBorrado) {
26              boolean eliminado = archivo.delete();
27              System.out.println("Resultado de la eliminación: " + eliminado);
28          } else {
29              System.out.println("Borrado cancelado por el usuario.");
30          }
31      }
32  }
```

Figura 10. Código fuente del Ejercicio 10 (Ejercicio10.java).

Resultado esperado:

- Si el archivo no existe, se crea y luego se elimina: Resultado de la eliminación: true.
- Con confirmarBorrado = false, el archivo se conserva y se muestra: Borrado cancelado por el usuario.

2.8 Habilidades blandas empleadas en la práctica

Liderazgo

Trabajo en equipo

☐Comunicación asertiva

La empatía

☒ Pensamiento crítico

Flexibilidad

La resolución de conflictos

☒Adaptabilidad

☒Responsabilidad

2.9 Conclusiones

La clase File permite crear, consultar y eliminar archivos de forma sencilla, y métodos como exists() y createNewFile() evitan sobrescribir o duplicar archivos que ya existen.

FileWriter y BufferedWriter facilitan el almacenamiento de datos en archivos de texto; el parámetro append=true conserva el contenido anterior, mientras que su omisión lo sobrescribe.

El manejo de excepciones con try-catch y el cierre de recursos con close() o try-with-resources hacen que los programas sean más seguros y eviten errores de lectura y escritura.

2.10 Recomendaciones

Verificar siempre la existencia del archivo y de su carpeta antes de escribir o eliminar,y cerrar los flujos al terminar para no perder datos.

Validar los datos ingresados por teclado (por ejemplo, con NumberFormatException) antes de calcular y guardar los resultados.

Subir los códigos a un repositorio de GitHub para mantener un respaldo y un historial ordenado de las prácticas.

2.11 Referencias bibliográficas

[1] Oracle, "Class File," Java Platformn, Standard Edition 8 API Specification. [En línea].Disponible: https://docs.oracle.com/javase/8/docs/api/java/io/File.html. [Accedido: nov.6, 2026].

[2] Oracle, "Class FileWriter," Java Platform, Standard Edition 8 API Specification. [En línea]. Disponible: https://docs.oracle.comm/javase/8/docs/api/java/io/FileWriter.html.[Accedido: nov. 6, 2026].

[3] Oracle, "Class BufferedWriter," Java Platform, Standard Edition 8 API Specification.[En línea]. Disponible: https://docs.oracle.com/javase/8/docs/api/java/io/BufferedWriter.html. [Accedido: nov.6,2026].

[4] Oracle, "The try-with-resources Statement," The Java Tutorials. [En línea]. Disponible:https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html.[Accedido: nov. 6, 2026].

2.12 Anexos

Repositorio de GitHub: https://github.com/Andyyyxx27/ficherosss